ECS 174 – Computer Vision
Final Project Report: Image Colorization with CNNs
Teammates: Zachary Foster, Ammond Wong, Sravya Kota
Date: 12/04/2025

1. Overview and Problem Description

In this project, we tackled the task of image colorization using convolutional neural networks (CNNs). The goal was to take a grayscale image as input and automatically predict a plausible color version of the same scene. Instead of directly predicting RGB values, we worked in the LAB color space, where the L channel encodes lightness, and the a/b channels encode chrominance (color information). We decided to use LAB over RGB because LAB separates lightness from color values, whereas RGB mixes color values in the respective channels with the lightness values. Our model receives the L channel and predicts the missing a and b channels.

Colorization is interesting because it sits at the intersection of low-level and high-level vision. The model needs to understand edges and textures, as well as the semantics of objects (e.g., sky, grass, and skin), to choose reasonable colors. The project allowed us to apply concepts from the CNN and deep learning lectures to a more open-ended, “creative” vision problem rather than standard classification.

2. Dataset and Preprocessing

For this project, we used the Tiny ImageNet-200 dataset for training and validation image sets. The data set consists of 200 different classes, 500 training images per class, and 50 validation images per class. The dataset is a smaller version of the ImageNet dataset and, as such, is more suitable for this project, which requires faster training with low availability of GPU power.

For each image, we first resized it to size 224 x 224, which is what our CNN model expects at the first layer. As each image comes in a RGB format, we converted each image from RGB to LAB using OpenCV. In the case of the training images, a random resize crop, horizontal flip, and color jitter were used to reduce overfitting and improve generalization.

After conversion:
We treated the L channel as our input “grayscale” image.
We used the a and b channels as ground truth targets for color.

We normalized the Lab channels to mirror the outputs of the sigmoid function [0,1] by dividing each channel by 255. This proved to be more stable than using tanh and a [-1,1] range for the ab channels.

During training, each sample therefore consisted of:
Input: L channel, stored as a tensor of shape (1, H, W).
Target: (a, b) channels, stored as a tensor of shape (2, H, W).

4. CNN Colorization Model

Our main model is a convolutional encoder–decoder network that directly predicts the a and b channels from the L channel. The architecture of the CNN makes use of the concept of reusing pretrained models taught in the lecture. It makes use of Pytorch's built-in Resnet18 trained using 'IMAGENET1K_V1' as a feature extractor due to its ability to extract meaningful semantic features, which would be useful for recolorising. It then feeds the feature map into a decoder to predict the a and b values. To modify the Resnet18 model to fit our use case, the first layer was modified to receive 1 channel (L) instead of 3 channels(RGB). The last 2 layers were also removed as they are required for the original classification task but not the current colorization task.

At a high level, the network works as follows:
Input: L channel of size 1×244×244.

Encoder:

Several convolutional layers with ReLU activations, followed by max pooling to gradually reduce the spatial resolution and increase the number of feature channels.
The encoder learns local patterns in the grayscale image, such as edges, corners, and textures, and combines them into higher-level features.

Decoder:

Upsampling layers via transpose-convolution, followed by batch norm layers and ReLU nonlinearities, were used to reconstruct a feature map back to the original spatial resolution. The Sigmoid function was used to ensure the predicted colors are within the normalized range of [0,1].

Formally, the model learns a function:
$f\theta : L \in R^{(1\times H\times W)} \rightarrow (\hat{a}, \hat{b}) \in R^{(2\times H\times W)}$

where θ denotes the network parameters. We trained the model using Mean Squared Error (MSE) loss between the predicted and ground truth a and b channels:
$L\_MSE = || \hat{a} a ||^2 + || \hat{b} b ||^2$

Because the output is dense (a value at every pixel), this is essentially a per-pixel regression problem rather than classification. We used the Adam optimizer with a learning rate of 1e−3.

We implemented the CNN in PyTorch and ran it in a Jupyter notebook on Google Colab. The main details of the setup are:

Batch size: 16
Optimizer: Adam
Loss function: MSE on the normalized a and b channels.
Number of epochs: Different versions of the model were saved, one for every 10 additional epochs it trained, up to a 50-epoch version

For evaluation, we considered two aspects:

Quantitative:

Compute the average MSE between predicted and ground truth (a, b) on the test set for the CNN model.

Qualitative:

Visualize the following for a sample of test images:

- Original RGB image (ground truth).
- Grayscale L channel (input).
- CNN colorization result.

The visual comparisons are important because small changes in MSE are not always easy to interpret visually, especially for human perception of color.

6. Results

The best version of the network was consistently the one trained for 20 epochs; after that, the model overtrained, causing washed-out colors and an oversaturation of red. The 20-epoch CNN model achieved an average test MSE of approximately 0.0030.

The output of the model, when visualised, looked similar to the initial colored image. Subplots include: the original image after it was converted to Lab space and back to RGB (To ensure that artifacts from the conversion process were fairly represented), and one recolor from each saved version of the network, except for the 50-epoch version, as it was massively overtrained.

Loss curves for the first four epochs are shown:

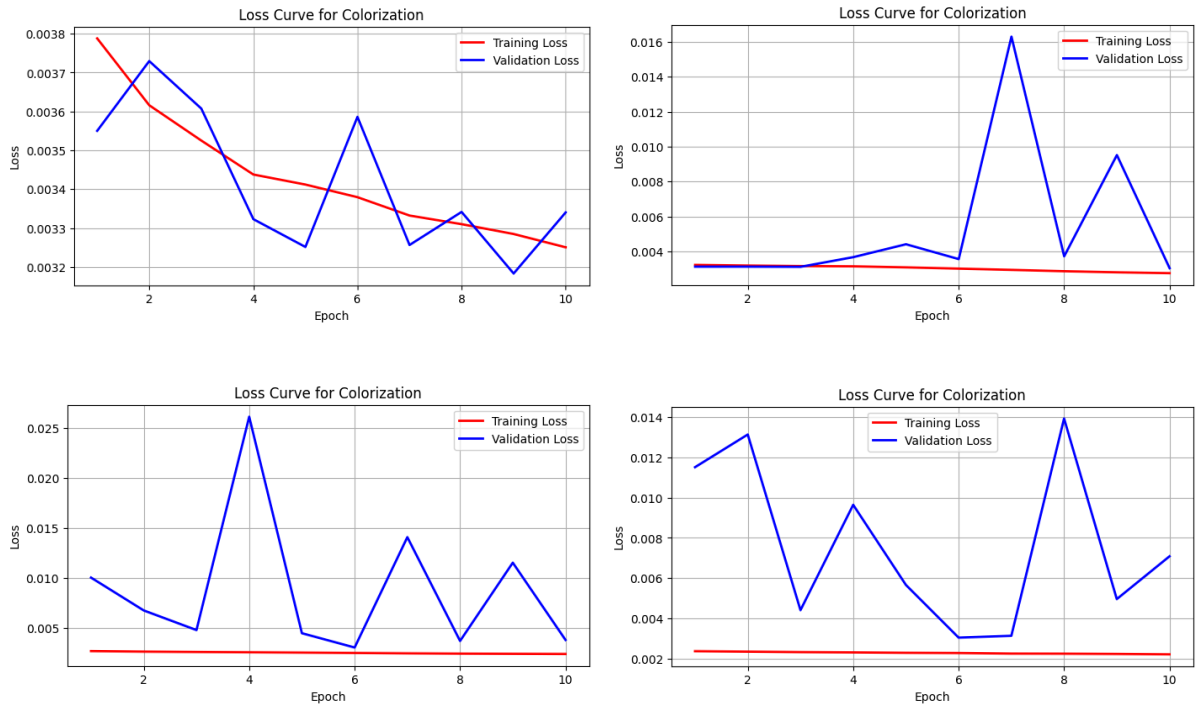

Figure 1. Loss curve graphs for epochs 1-10 (top left), 11-20 (top right), 21-30 (bottom left), 31-40 (bottom right)

7. Discussion

The CNN can process local neighborhoods of the image using convolutional filters, similar to the classification networks we studied in HW2. This allows it to:

Learn edge and texture features that help distinguish different regions.

Use deeper layers to combine these local features into higher-level representations that correlate with specific object categories or materials.

Predict color patterns that depend on both local texture and larger-scale structure.

However, colorization is still a very ambiguous problem. There are many valid ways to color the same grayscale image, and our network is only approximating one of them. It is possible to pick colors that are technically wrong but still visually plausible. Training on a small, low-resolution dataset like Tiny ImageNet also limits the overall generalisation and ability to predict a and b values for the decoder.

Another limitation is that we treated colorization as a simple regression problem with MSE loss. While MSE is convenient, it tends to penalize large deviations strongly and can encourage the network to predict “average” colors, which may contribute to slightly desaturated or less vibrant outputs. More advanced approaches use perceptual losses or generative adversarial networks (GANs) to produce more vivid, realistic colorizations, but those were beyond the scope of this project.

Figure 2. Representations of images recolored by the network at various levels of training.

8. Conclusion and Future Work

In this project, we implemented and evaluated a CNN for image colorization:

If we had more time and computational resources, some natural next steps would include:

- Training on higher resolution images and a larger, more diverse dataset.
- Using data augmentation to improve generalization.
- Exploring different loss functions, such as perceptual losses or adversarial training, to encourage more vivid and realistic colors.
- Adding class conditional information (e.g., using labels to guide color choices for certain objects).
- Using a larger pre-trained model like the full ResNet model.

Overall, the project made the ideas from the CNN and deep learning lectures feel much more concrete, because we could see how the model's learned filters and training dynamics translate into visible differences in generated images.

9. Contributions

Although this was a group project, the workload was not perfectly balanced. Roughly, the contributions were:

Teammate 1:Zachary Foster

- Edited NN multiple times to ensure a working model
- Trained multiple neural networks
- Created code so the trained model can output images
- Revised project report

Teammate 2: Ammond Wong

- Wrote code for the project
- Wrote descriptions and comments of code in the Jupyter notebook
- Wrote project report

Teammate 3: Sravya Kota

- Implemented the project proposal
- Wrote the project report
- Implemented the slides and edited the video
- Helped in debugging the code
- Helped in dividing work among people